

Mock Interview Platform Documentation

1. Project Overview

This website helps student candidates prepare for technical corporate placement interviews.

Candidates pay a platform deposit fee and broadcast a request detailing their upcoming target company round.

HR Professionals (Interviewer Experts) view the global market pool and claim open requests to match with a candidate.

Admins monitor the platform safety metrics to make sure no student request is left stranded or waiting for more than 48 hours.

2. Platform Architecture (How Data Flows)

The platform is split into three main components:

Frontend User Interfaces: Simple HTML and CSS dashboards (candidate-dashboard.html, hr-dashboard.html, admin-dashboard.html) where users view and interact with data.

Backend Server Engine: A Node.js and Express application (server.js, auth.js, requests.js) that handles API requests, matches parameters, and processes claims.

Database Asset Layer: A MongoDB instance running locally that holds secure data entry configurations for accounts and scheduled actions.

3. Database Collection Blueprints

The system utilizes two main database blueprints (schemas) managed via Mongoose inside MongoDB:

A. The `users` Collection (`user.js`)

Stores account access records and identity parameters.

email: The lowercase unique electronic mail identity used to log in (e.g., sai@example.com).

password: A secure, encrypted text string processed using bcryptjs.

role: Defines what the user can see. It must be either 'candidate', 'hr', or 'admin'.

company: The corporate workspace name (Only filled out if the user is an HR expert).

B. The requests Collection (Request.js)

Tracks the real-time lifecycle of an interview broadcast.

candidateId: Connects back to the specific user document who created the request.

candidateName: Display name of the student candidate (e.g., "Sai").

companyApplied: Target corporate selection round (e.g., "Infosys Springboard").

stream: Selected domain stream focus (e.g., "Data Engineering", "Software Development").

interestedSlot: The date and time the student wants their mock round scheduled.

originalSlot: The student's actual company placement interview date.

status: Current tracking state. Automatically starts as 'pending'. Changes to 'accepted' when an HR matches, and 'completed' after feedback is given.

interviewerId: Connects to the specific HR user who claimed the session.

meetLink: Automatically generated Google Meet virtual room execution token string.

feedback: Custom performance score notes saved after the round finishes.

4. User Workspace Workflows

Candidate Workflow

Login: A student logs in. The system saves their credentials locally in the browser memory (localStorage).

Submit Request: The student fills out the request form, selects their target stream (like *Data Engineering*), pays the escrow token fee, and hits submit.

Broadcast: The dashboard triggers an HTTP POST message to `/api/requests/create` saving the document to MongoDB with a status of "pending".

History Tracking: The candidate can view their open entries under "Requested Interviews" and check for confirmed matches inside "Upcoming Interviews".

HR Expert Workflow

Live Synchronization Checking: When an HR expert logs into their control dashboard, an automated heartbeat script runs every 5 seconds checking the backend `/api/requests/pool` route.

The Open Market Pool: The dashboard displays any candidate document currently matching status: "pending".

Claiming a Session: When the HR clicking the "Accept & Claim Session" button, a secure PUT request is dispatched to `/api/requests/claim/:id`.

Schedule Update: The backend alters the status to "accepted", registers the HR's internal account ID onto the slot document, and provides a mock Google Meet execution endpoint. The card instantly moves into the **Confirmed Scheduled Sessions** tracker table.

Admin Safety Queue Workflow

Stuck Requests Monitoring: The main admin dashboard automatically checks the `/api/requests/admin/stuck` engine endpoint every 5 seconds.

The 48-Hour Safety Trigger: The backend analyzes timestamps to check for any items where `status === 'pending'` and the record's creation date (`createdAt`) is older than 48 hours.

Escalation Alert Banners: If a request is stuck, a bright red alert card surfaces on the root admin panel deck, allowing administrators to execute high-priority manual corporate assignment routing overwrites.

5. Local Setup and Deployment Guide

To get the entire digital platform running locally on your computer:

Start MongoDB: Open MongoDB Compass and ensure a local database service is active on standard string port `mongodb://localhost:27017`.

Install Workspace Dependencies: Open your terminal inside the project folder and install the necessary libraries:

Bash

```
npm install express mongoose cors bcryptjs jsonwebtoken
```

Boot Up the Backend Server: Execute the initialization command:

Bash

```
node server.js
```

Launch Dashboards: Open your frontend dashboard files directly inside your internet browser window to perform a test run!